

手眼标定流水线 真机测试任务清单

RealMan Robot Arm — ROS2 Workspace

Hand-Eye Calibration Pipeline Hardware Test Plan

文档版本: V1.0
日期: 2026-06-30
编制: Sisyphus (AI Agent)
审阅: Lorenzo Feng
状态: 待执行
密级: 内部

RealMan 手眼标定流水线 — 真机测试任务清单

目标：对 `realman_driver`、`realman_vision`、`realman_calibration` 三个包在真机上完成功能验证，确认 TCP 通信、运动控制、相机采集、标定流水线的每一环节可正常工作。

测试环境要求

项目	要求
机械臂	RealMan RM65（或其他 RM 系列），已开机，TCP 可通
机械臂 IP	默认 192.168.1.18（或启动时 <code>--ip</code> 指定）
相机	Intel RealSense D400 系列，USB 3.0 连接
标定板	打印好的棋盘格（11×8 内角点，方格边长 30mm）或 ChArUco 板
工作区	~/ws/realman，已 <code>colcon build</code> 通过
ROS2 环境	<code>source /opt/ros/humble/setup.bash</code> && <code>source ~/ws/realman/install/setup.bash</code>

Phase 1: 基础硬件连通性

T1.1 — Arm TCP 连接

项	内容
操作	使用 SDK demo 或写一个最小程序连接机械臂
预期	连接成功， <code>rm_create_robot_arm</code> 返回有效 <code>handle</code> ， <code>rm_get_current_arm_state</code> 可读关节角度
通过标准	连接不报错，能读到当前关节位置（角度值在合理范围）

快速验证：启动 `gripper_test`（它也会连接 `arm`）

```
ros2 run realman_driver gripper_test
```

或跑一次 `collect_data`（-h 看参数）

```
ros2 run realman_calibration collect_data --ip 192.168.1.18 --count 1
```

如果连接失败，检查：机械臂 IP 是否正确、端口 8080 是否开放、防火墙/网络是否在同一网段。

T1.2 — RealSense 相机采集

项	内容
操作	运行任一采集程序，确认能拿到彩色帧
预期	<code>CameraStream::next()</code> 返回非空 <code>CameraFrame</code> ， <code>color</code> 图像尺寸正常（如 640×480）
通过标准	连续取 100 帧无空帧，帧率稳定

用 `collect_data` 交互模式测试相机

```
ros2 run realman_calibration collect_data --ip 192.168.1.18 --output /tmp/test_cam --count 5
```

程序启动后会显示相机预览窗口（需要 `DISPLAY`），按 `Q` 退出

Phase 2: 机械臂运动与状态读取

T2.1 — JointPosition 读取正确性

项	内容
操作 预期 通过标准	将机械臂手动拖到 3 个不同的已知姿态，每次读取 jointPosition() 读数稳定（波动 < 0.01 rad），与视觉观察的关节位置一致 3 次读数均为有效弧度值（范围 $\pm\pi$ ），无 0.0 全体报告

T2.2 — toolPose() 读取正确性

项	内容
操作 预期 通过标准	机械臂停在已知位置，读取末端 TCP 位姿 x/y/z 米级合理，roll/pitch/yaw 弧度合理 位姿数值与机械臂示教器显示一致。注意单位换算：代码中为弧度+米，示教器通常为 $\text{rad} \approx 57.3^\circ$ ，1 m = 1000 mm)

T2.3 — moveJ 弧度→度转换验证（关键！）

项	内容
操作 预期 通过标准	发送一个仅动 Joint1 的简单指令，观察实际运动方向和幅度 Joint1 旋转 0.5 rad（约 28.6° ），机械臂正确运动对应角度 运动后读取 jointPosition()，Joint1 变化约 0.5 rad，其他关节不变

高危点：C SDK 使用度，我们的 API 使用弧度。如果转换错误，机械臂可能以错误的幅度运动或报错。

T2.4 — stop() 急停

项	内容
操作 预期 通过标准	机械臂运动中（moveJ 大角度），调用 stop() 机械臂立即停止，不报错 stop() 调用后 1 秒内机械臂完全停止

T2.5 — moveL 直线运动

项	内容
操作 预期 通过标准	对机械臂当前 TCP 位姿加小幅偏移（如 Z+50mm），调用 moveL() 机械臂末端沿直线从当前点移动到目标点 运动轨迹在示教器上显示为直线

Phase 3: 夹爪操作

T3.1 — 夹爪基本开合

项	内容
操作	<code>gripper(1000) → gripperRelease(500) → gripper(1000)</code> 循环 3 次
预期	夹爪完全闭合、完全张开
通过标准	3 次循环均正常，无超时或报错

T3.2 — `gripperPick` 力控抓取

项	内容
操作	在夹爪中放入物体，调用 <code>gripperPick(speed=500, force=200)</code>
预期	夹爪闭合直到达到力阈值，不损坏物体
通过标准	抓取成功， <code>gripperState()</code> 显示状态正常

T3.3 — `gripperState()` 读取

项	内容
操作	分别读取夹爪全开、半开、全闭 3 个状态
预期	<code>actpos</code> 字段反映实际开口位置
通过标准	3 个状态的值递增/递减一致

Phase 4: 棋盘格检测验证

T4.1 — 真实相机棋盘格检出率

项	内容
操作	启动采集程序交互模式，将打印好的棋盘格（11×8 内角点）放在相机视野不同位置和角度
预期	棋盘格角点被正确检测（绿色圆点 <code>overlay</code> ），至少覆盖正面、倾斜 30°、倾斜 60°
通过标准	10 个不同角度中 ≥8 个成功检出
前提	T1.2 通过

```
ros2 run realman_calibration collect_data --ip 192.168.1.18 --output /tmp/calib_test --count 18
# 交互模式：移动棋盘格，观察 overlay 绿色角点是否稳定
# 看到绿色角点 → 按 S 保存；红色叉号表示未检测到
```

Phase 5: 数据采集集成

T5.1 — 交互模式完整采集

项	内容
操作	机械臂连接 + 相机连接 + 棋盘格就绪，交互模式采集 18 张图片
预期	每次按 S 后，image/ 下生成图片，robot_poses.csv 追加一行位姿数据
通过标准	18 张图片全部保存，robot_poses.csv 有 18 行 + header，旋转多样性门控通过（采集自然结束时提示“Rotation diversity OK”）
输出产物	images/00001.jpg ... 00018.jpg, robot_poses.csv

T5.2 — 旋转多样性门控验证

项	内容
操作	故意只在相似角度采集（如机械臂不动、只平移棋盘格），确认系统拒绝结束
预期	系统提示“WARNING: rotation diversity insufficient”
通过标准	提示信息清晰，可按 Q 强制退出

Phase 6: 全流水线端到端

T6.1 — run_pipeline 一键标定

项	内容
操作	在采集好数据后（或直接用交互模式自然结束），运行完整流水线
预期	4 个阶段依次执行，最终产出 calibration_result.yaml
通过标准	所有阶段不报错，输出含 R (3×3)、t (3×1)、reprojection_error、method、c

方式 A: 交互模式跑全流程（采集 + 标定一步完成）

```
ros2 run realman_calibration run_pipeline \
  --ip 192.168.1.18 \
  --output /tmp/calib_test \
  --count 18 \
  --mode in_hand \
  --board-w 11 --board-h 8 --square-size 0.03
```

方式 B: 分步执行

```
ros2 run realman_calibration collect_data --ip 192.168.1.18 --output /tmp/calib_test --count 18
ros2 run realman_calibration calibrate_camera --output /tmp/calib_test --board-w 11 --board-h 8
ros2 run realman_calibration compute_hand_eye --output /tmp/calib_test --mode in_hand
```

T6.2 — 标定结果质量评估

项	内容
操作 通过标准	检查 calibration_result.yaml 内容 rotation_matrix — 3×3, $\det \approx 1$ (误差 < 0.01) translation_vector — 3×1, 值在合理范围 ($< 2\text{m}$) reprojection_error — < 100.0 px condition_number — > 0 且 < 1000 method — 为 “Tsai” / “Park” / “Horn” / “Daniilidis” 之一 目测验证: R 矩阵应近似单位阵 (若相机固定在夹爪附近正方向), t 应接近相机到法兰的物理距离

```
cat /tmp/calib_test/calibration_result.yaml
```

Phase 7: ROS2 节点集成

T7.1 — calib_node 启动

项	内容
操作 预期 通过标准	启动 calib_node 并检查日志 节点启动, 打印 “CalibNode ready — arm=... mode=... board=...” ros2 node list 可以看到 /calib_node

```
ros2 run realman_calibration calib_node --ros-args -p arm_ip:=192.168.1.18
```

T7.2 — 服务调用

项	内容
操作 通过标准	通过 ros2 service call 逐个调用 4 个服务 ~/calibrate → 成功返回相机标定结果 (重投影误差、使用图像数) ~/hand_eye → 成功返回手眼标定结果 (方法、重投影误差) ~/run → 成功返回完整流水线结果 (需先采集数据) ~/stop → 机械臂急停, 返回 success=true

```
ros2 service call /calib_node/calibrate std_srvs/srv/Trigger "{}"
ros2 service call /calib_node/hand_eye std_srvs/srv/Trigger "{}"
ros2 service call /calib_node/stop std_srvs/srv/Trigger "{}"
```

T7.3 — TF 广播验证

项	内容
操作 预期	在 calib_node 完成标定后, 检查 TF 树 base_link → camera_link 的静态变换被广播

项	内容
通过标准	ros2 run tf2_tools view_frames 生成的 PDF 包含该变换链或 ros2 run tf2_ros tf2_echo base_link camera_link 有输出

在 calib_node 完成 ~/hand_eye 或 ~/run 之后：
ros2 run tf2_ros tf2_echo base_link camera_link

验收总结

必须通过（阻塞发布）

- ☐ T1.1 — Arm TCP 连接
- ☐ T1.2 — RealSense 相机采集
- ☐ T2.1 — jointPosition 读取
- ☐ T2.2 — toolPose 读取
- ☐ T2.3 — moveJ 弧度/度转换
- ☐ T2.4 — stop 急停
- ☐ T4.1 — 棋盘格检出率 $\geq 80\%$
- ☐ T5.1 — 18 张完整采集 + 旋转多样性通过
- ☐ T6.1 — 全流水线产出 calibration_result.yaml
- ☐ T6.2 — 标定结果质量达标

建议通过（影响体验）

- ☐ T2.5 — moveL 直线运动
- ☐ T3.1 — 夹爪基本开合
- ☐ T3.2 — gripperPick 力控
- ☐ T3.3 — gripperState 读取
- ☐ T5.2 — 旋转多样性门控提示
- ☐ T7.1 — calib_node 启动
- ☐ T7.2 — ROS2 服务调用
- ☐ T7.3 — TF 广播
- ☐ T8.1 — V1 stub 行为确认

备注

1. **弧度/度量纲**：代码中所有角度使用弧度、长度使用米。与示教器对比时需注意单位换算（ $1 \text{ rad} \approx 57.3^\circ$ ， $1 \text{ m} = 1000 \text{ mm}$ ）。
2. **棋盘格打印**：内角点 11×8 ，方格边长 30mm（与默认参数匹配）。可修改 --board-w / --board-h / --square-size 适应不同标定板。
3. **headless 运行**：若无显示器（SSH 远程），采集程序会自动降级为 headless 模式，在终端输出状态日志。
4. **报错处理**：任何步骤如果遇到报错，记录完整错误信息（包含错误码），方便定位。
5. **测试数据**：保留每次测试产出的文件，归档到服务器：
 - images/（采集图片）
 - robot_poses.csv（位姿记录）
 - calibration_result.yaml（标定结果）